

Track 13 — Der unauthentifizierte Perimeter

Datum: 28.08.2026

Vorgänger: Track 12 (Abrechnung & Finanzflüsse), HEAD f4231e6

Datenbank: Supabase-Projekt nnwykttkqibdjxgimjyuq, 310 public-Tabellen, 404 Migrationsdateien im Repo

Die Angriffsfläche — und warum sie neu ist

Die Tracks 1 bis 12 haben durchweg einen angemeldeten Akteur vorausgesetzt. Die Fragen lauteten: Darf dieser Nutzer diese Zeile sehen (Tracks 6, 7, 10)? Darf er sie schreiben (Tracks 5, 9)? Stimmt der Betrag, den er schreibt (Track 12)? Und was passiert mit seinen Daten, wenn er geht (Track 11)?

Track 13 stellt die Frage davor: Was kann jemand ohne Konto?

Das ist keine Verfeinerung der bisherigen Fragen, sondern eine andere. Ein Zugriffsaudit prüft Berechtigungen — aber am Perimeter gibt es keine Berechtigung zu prüfen. 22 Routen dieser Anwendung sind absichtlich ohne Anmeldung erreichbar (Liste unter „Methode“): das Kontaktformular, die Lead-Erfassung, der Newsletter samt Abmeldung, fünf Wege der Besucher- und Conversion-Messung, der Beratungs-Chat, drei Webhooks, der Passwort-Reset und einige reine Auskunftsrouten. Sie sind kein Versehen — die öffentliche Website braucht sie.

Der entscheidende Befund über diese Fläche ist eine Negativ-Feststellung, und sie bestimmt den ganzen Track:

> anon hat auf keiner der 310 public-Tabellen ein Schreibrecht. RLS ist auf allen 310 aktiv. Keine der 362 public-Funktionen ist zugleich SECURITY DEFINER und für anon ausführbar.

Die Datenbank ist gegen den anonymen Aufrufer also dicht. Daraus folgt aber nicht Entwarnung, sondern eine Verschiebung: Alle diese Routen schreiben mit dem Dienstschlüssel. Der Dienstschlüssel kennt keine Policies. RLS sieht diese Anfragen nie. Der einzige Riegel zwischen einem beliebigen Fremden im Internet und der Produktionsdatenbank ist der Quelltext der Route selbst.

Genau dort liegen die Befunde dieses Tracks — mit einer Ausnahme, und die ist die schwerste.

Methode

Ein nur lesendes Orakel gegen die Produktion. `public._run_sql` mit abschließendem `RAISE EXCEPTION`: die Transaktion rollt immer zurück, das Ergebnis kommt über die Fehlermeldung. Damit sind Rechte (`has_table_privilege`, `has_function_privilege`), alle 984 Policies, Constraints, Fremdschlüssel und Bestandszahlen abfragbar, ohne etwas zu verändern.

Eine Schreibprobe unter fremder Rolle — im selben Rollback. Für Befund B1 genügte die Policy-Lesung nicht: die Frage war, ob die Kette aus Grant, permissiver Policy, restriktiver `org_fence` und Spalten-Default **tatsächlich** durchlässt. Also `SET LOCAL ROLE authenticated; INSERT ...;` innerhalb des immer abbrechenden DO-Blocks. Nichts wird festgeschrieben — die Kontrollabfrage danach zeigt unverändert 32 Zeilen und 0 Probenzeilen.

Ein anon-Zugriffstest über HTTP mit dem öffentlichen Schlüssel gegen acht Perimeter-Tabellen — 200 mit leerem Rumpf ist mehrdeutig, deshalb wird der Rumpf immer mitgemeldet.

Quelltextlesung aller 22 anonym erreichbaren Routen sowie der Module, an denen sie hängen (`lib/rate-limit-persistent.ts`, `lib/api/cron-auth.ts`, `lib/dsgvo/loeschkatalog.ts`). Die Liste, damit die Zahl nachprüfbar ist:

- Schreiben mit Dienstschlüssel: `kontakt`, `lead-inquiry`, `newsletter`, `newsletter/unsubscribe`, `track`,

track/page-view, track-conversion, analytics/vitals, analytics/capi, visitor-alert, beratung-chat, auth/send-reset, auth/check-rate-limit, coach/anfrage

- Webhooks (signaturgeprüft): stripe/webhook, whatsapp/webhook, coach/webhook
- Nur Auskunft: pricing/calculate, google-reviews, client-ip, health, expansion/status, coach/tarife

Eine eigene Methodenkorrektur, festgehalten: Die erste Routen-Liste entstand durch ein Grep nach Guard-Namen und meldete 252 von 412 Routen als ungeschützt. Das war falsch. Die Guards liegen überwiegend hinter Fach-Wrappern (requireOpsAdmin, requirePersonalAdmin, authorize()), die das Grep nicht kannte. Die tatsächlich anonyme Fläche sind 22 Routen. Ein Audit, das mit 252 Falschbefunden startet, findet die 22 echten nicht mehr — die Korrektur stand am Anfang, nicht am Ende.

Und die Korrektur war beim zweiten Mal nicht vollständig. Der verfeinerte Guard-Filter meldete 55 Kandidaten, die von Hand durchgesehen wurden; die meisten rufen auth.getUser() ohne benannten Guard und sind damit doch geschützt. Dabei fiel /api/track/page-view zunächst durch — sie war in der ersten Befundfassung von B2 und B5 nicht enthalten und kam erst beim Nachzählen dazu. Sie ist ausgerechnet der größte Fall beider Befunde (8.315 Zeilen, 2.033 verschiedene IP-Adressen). Das steht hier, weil es die Grenze der Methode zeigt: die Fläche wurde durch wiederholtes Filtern eingegrenzt, und jede Runde hat noch etwas gefunden. Eine dritte Runde könnte etwas finden, das auch diese übersehen hat.

Befunde

B1 (P1, LIVE VERIFIZIERT) — Die einzige offene Tür der Datenbank führt ins CRM

lead_inquiries trägt eine Policy:

Anyone can submit lead inquiry | INSERT | PERMISSIVE | roles=public | WITH CHECK true

Sie stammt aus 20260606_lead_inquiries.sql und steht dort unter dem Kommentar `*„Öffentliches Insert (Website-Formular, kein Auth nötig)“*`. Geschrieben wurde sie für einen Entwurf, in dem der Browser die Zeile mit dem öffentlichen Schlüssel selbst anlegt.

So läuft es nicht mehr. POST /api/lead-inquiry schreibt mit dem Dienstschlüssel und umgeht RLS vollständig. Die Policy gewährt der Anwendung also nichts, was sie braucht. Und sie gewährt es auch nicht mehr denen, für die sie gedacht war: anon hat auf lead_inquiries kein INSERT-Grant.

Wirksam ist sie ausschließlich für authenticated — eine Rolle, die in ihrem Kommentar nicht vorkommt.

Live nachgestellt (28.08.2026, in der immer zurückrollenden Transaktion):

Rolle	Ergebnis
authenticated	ERFOLGREICH
anon	abgewiesen: 42501 permission denied
Bestand danach	32 Zeilen, 0 Probenzeilen

Ein systematischer Durchlauf durch alle 984 Policies zeigt: Dies ist die einzige permissive INSERT-Policy des gesamten Schemas mit WITH CHECK true für public. Es gibt keine zweite offene Tür — es gibt genau diese.

Was das heißt. Jedes angemeldete Konto — ein Kunde, ein Engel, ein Fahrer, ein frisch selbst registriertes Konto — kann per PostgreSQL beliebige Zeilen in die Lead-Pipeline des Betreibers schreiben und umgeht damit jede Schranke, die /api/lead-inquiry aufgebaut hat:

- rateLimitPersistent('lead:\${ip}', 5, 10 min)
- das Honeypot-Feld website

- die Längenkappen (name 120, phone 40, message 2000, ...)
- die Plausibilitätsprüfung „mindestens 6 Ziffern" auf phone
- die PLZ-Formprüfung

organization_id trägt den Spalten-Default current_org_id(). Diese Funktion ist fail-open: ein Konto ohne Zeile in organization_members landet in der Stamm-Organisation — also genau dort, wo die 32 echten Leads liegen. Die RESTRICTIVE org_fence ist damit von selbst erfüllt und keine Schranke.

Schreibbar sind auch die internen Bearbeitungsfelder: status, notes, assigned_to, follow_up_date, converted_client_id.

Der zweite Teil desselben Befundes: lead_inquiries.status trägt live gar keine Bedingung — die Tabelle hat außer Primärschlüssel und drei Fremdschlüsseln keinen einzigen CHECK. Die CRM-Oberfläche (app/mis/crm/page.tsx, LEAD_STATUS) arbeitet mit genau fünf Werten und gruppiert die Pipeline-Tafel danach. Ein freier Wert erzeugt eine Spalte, die niemand sieht: der Lead ist unsichtbar, ohne gelöscht zu sein.

Behoben durch Migration 20261018000000 (eingecheckt, nicht angewendet, Rollback 20261018000001): die Policy entfällt, status bekommt einen CHECK über die fünf Werte der Oberfläche. Bestandsprüfung vorab: 0 von 32 Zeilen verletzen die Bedingung.

Ausdrücklich nicht getan: eine Ersatz-Policy für anon. Eine solche wäre nur richtig, wenn das Formular wieder direkt aus dem Browser schriebe — und dann fielen alle oben aufgezählten Schranken der Route ebenfalls weg.

Warum P1 und nicht P0: Es entsteht kein Lesezugriff auf fremde Daten. Ein nicht-Admin bekommt aus lead_inquiries weiterhin null Zeilen (die einzige permissive SELECT-Policy verlangt is_admin()), und anon kommt gar nicht heran. Der Schaden ist das Einschleusen in ein Geschäftssystem, nicht der Abfluss aus ihm.

B2 (P2) — Der halbe Rate-Limit-Umbau

Am 19.08.2026 wurde rateLimitPersistent gebaut, mit einer ausdrücklichen Begründung im Kopf der Datei: *auf Vercel startet jede neue Serverless-Instanz mit leerem Zähler; für den unauthentifizierten /api/visitor-alert (schreibt mit Admin-Client, versendet Mail, legt Notifications an) reicht das nicht.*

Sieben Routen wurden damals umgestellt. Fünf wurden übersehen — und es sind genau die, auf die dieselbe Begründung wörtlich zutrifft:

Route	Zähler vorher	Schreibt mit Dienstschlüssel in
/api/track	eigene Map im Modul-Scope	visitors, visitor_locations + löst Mailversand aus
/api/track/page-view	rateLimit() aus lib/rate-limit.ts	page_views (mit voller IP)
/api/track-conversion	eigene Map im Modul-Scope	conversions (mit voller IP)
/api/analytics/vitals	eigene Map im Modul-Scope	analytics_events
/api/analytics/capi	rateLimit() aus lib/rate-limit.ts	— (Welle-1-Stub)

Drei davon hatten sich sogar ihren eigenen Zähler gebaut, statt lib/rate-limit.ts zu benutzen. Ein Zaun, der nur den Namen rateLimit sucht, hätte sie nicht gefunden.

/api/track/page-view ist der bitterste Fall. Ihr Dateikopf beschreibt sie ausdrücklich als Ersatz für einen früheren Direktschreibpfad aus dem Browser, dessen Policy WITH CHECK (true) für public lautete — *„jeder Unbeteiligte konnte die Tabelle unbegrenzt befüllen"*. Als erste der drei neuen Schranken nennt der Kopf: *„Rate-Limit pro IP (Bot-Floods, Doppel-Submits)"*. Genau diese Schranke war instanzlokal und damit auf Vercel keine. Die Route hat eine offene Tür geschlossen und dabei die Zusage, die sie an ihre Stelle setzte, nicht eingehalten.

Dass der persistente Zähler funktioniert und im Einsatz ist, ist live belegt: `api_rate_limit_hit` ist SECURITY DEFINER mit festem `search_path`, `api_rate_limits` trägt 70 Zeilen über fünf Präfixe (`visitor-alert 62`, `beratung-chat 2`, `lead 2`, `send-reset 2`, `kontakt 1`). Keines dieser Präfixe gehört zu den fünf Routen — sie haben nie mitgezählt.

Behoben: alle fünf auf `rateLimitPersistent` gezogen. `/api/analytics/capi` bewusst mit, obwohl sie heute nichts persistiert: in Welle 2 soll sie an Meta und TikTok POSTen, dann ist das Limit ein Kostendeckel gegen eine fremde API — umgestellt, solange es noch nichts kostet.

Zusätzlich entfiel in `/api/track` eine `setInterval`-Aufräumschleife im Modul-Scope einer Serverless-Funktion.

B3 (P2) — Die Newsletter-Abmeldung ohne jeden Nachweis

`GET /api/newsletter/unsubscribe?email=<adresse>` nahm die Adresse aus der Query und meldete ab. Ohne Token, ohne Signatur, ohne Ratenbegrenzung, mit dem Dienstschlüssel.

Daraus folgten drei verschiedene Dinge mit drei verschiedenen Ursachen:

1. Fremdbemeldung. Wer eine Adresse kennt oder rät, meldet sie ab. Der Verteiler ist aus der Ferne leerbar, und der Betreiber merkt nichts — eine Abmeldung ist ein völlig normaler Vorgang.
2. Der Automat meldet ab. Ein GET-Link in einer Mail wird nicht nur von Menschen geöffnet. Sicherheitsprodukte im Mailweg (Link-Umschreibung, Vorabprüfung von Zielen, Bild-Proxys) rufen Links beim Zustellen auf. Der Empfänger ist dann abgemeldet, ohne je geklickt zu haben. Genau deshalb verlangt RFC 8058 für die Ein-Klick-Abmeldung ein POST.
3. Kein Wirkungsnachweis. `.update()` ohne `.select()` meldet in PostgreSQL keinen Fehler, wenn null Zeilen getroffen wurden. Die Seite sagte „Sie wurden erfolgreich abgemeldet“ auch dann, wenn nichts geschehen war — dieselbe Klasse wie Track 11 B5.

Behoben durch `lib/newsletter/abmelde-token.ts` (HMAC-SHA256 über die normalisierte Adresse) und eine umgebaute Route: GET zeigt nur eine Bestätigungsseite und fasst die Datenbank nicht an, POST meldet ab, beide verlangen ein gültiges Token, der Schreibweg trägt `.select('id')` und eine persistente Ratenbegrenzung.

Warum HMAC und keine Token-Tabelle. Die Widerrufs-Token der Kontolöschung liegen in `account_deletion_tokens`, weil sie einmal verwendbar sein und ablaufen müssen. Für eine Abmeldung gilt beides ausdrücklich nicht: der Link muss noch in einer zwei Jahre alten Mail funktionieren (Art. 21 DSGVO — der Widerspruch darf nicht erschwert werden) und beliebig oft benutzbar sein.

Warum ausdrücklich nicht fail-closed beim Schlüssel. Ein eigener `NEWSLETTER_ABMEDELDE_SECRET` hat Vorrang; fehlt er, wird ein Schlüssel aus dem Dienstschlüssel abgeleitet (HMAC über eine feste Kennung — der Dienstschlüssel selbst signiert nie und ist aus dem Token nicht rekonstruierbar). Fail-closed hieße hier: ohne gesetzte Variable ist gar keine Abmeldung mehr möglich. Fail-closed ist die richtige Antwort auf „darf jemand mehr, als er soll“ — nicht auf „kann jemand ein Recht ausüben, das ihm zusteht“.

Altlinks: Mails von vor dieser Änderung tragen `?email=` ohne Token und funktionieren nicht mehr. Vertretbar, weil `newsletter_subscribers` live 0 Zeilen führt — es gibt keinen Empfänger, dessen Altlink auflaufen könnte. Wäre der Verteiler belegt, wäre ein Übergangsfenster der richtige Weg; das steht als Kommentar in der Route.

B4 (P2, LIVE VERIFIZIERT) — Die Löschung entfernt das Etikett, nicht die Person

`visitor_locations.user_id` trägt einen Fremdschlüssel auf `profiles(id)` mit `ON DELETE SET NULL`. Bei der endgültigen Kontolöschung wird die Spalte also genullt, und die Zeile bleibt stehen.

Stehen bleiben damit auch: `ip_address`, `user_agent`, `page_path`, `portal`, `created_at`.

Eine volle IP-Adresse ist nach Art. 4 Nr. 1 DSGVO selbst ein Personenbezug. SET NULL entfernt das Etikett, nicht die Kennung. Was bleibt, ist eine pseudonymisierte Zeile, keine gelöschte — und der Inhalt ist die seitenweise Bewegungsspur einer Person durch ein Pflegeportal.

Dasselbe gilt fuer page_views.user_id — Fremdschluessel auf auth.users, ebenfalls ON DELETE SET NULL, und mit Abstand der groessere Bestand.

Live am 28.08.2026:

	visitor_locations	page_views
Zeilen gesamt	3.850	8.315
davon mit user_id	578	1.111
verschiedene Konten	38	43
davon mit voller IP	2.743	6.632
verwaiste user_id	0	—

284 der visitor_locations-Zeilen tragen das Portal kunde.

Der eigentliche Punkt ist aber nicht die Zahl, sondern die Abwesenheit einer Entscheidung: Der Löschkatalog aus Track 11 — die Datei, die es ausdrücklich gibt, damit „was bleibt und was geht“ eine Entscheidung ist und kein Vergessen — führte keine der sieben Perimeter-Tabellen. Niemand hat je entschieden, dass diese Zeilen überleben sollen; sie sind nie zur Sprache gekommen.

Behoben: je ein Eintrag im Löschkatalog für visitor_locations.user_id und page_views.user_id, Entscheidung loeschen, mit Begründung, warum aufbewahren hier eine falsche Aussage wäre — der Katalog definiert aufbewahren als „der Personenbezug fällt über SET NULL weg“, und genau das trifft hier nicht zu.

B5 (P2, LIVE VERIFIZIERT) — Volle IP-Adressen, unbefristet, neben einer ungenutzten Hash-Spalte

Fünf Routen der öffentlichen Website schreiben ohne Anmeldung mit dem Dienstschlüssel. Was dabei entsteht, hat nie jemand wieder angefasst:

Tabelle	Zeilen	mit voller IP	verschiedene IPs	älteste
page_views	8.315	6.632	2.033	26.03.2026
visitor_locations	3.850	2.743	1.091	15.03.2026
visitors	3.391	3.391	548	09.03.2026
conversions	38	38	26	16.04.2026
analytics_events	4.170	0 (ip_hash)	—	12.08.2026

vercel.json führte neun Cron-Einträge. Keiner räumte hier auf. Es gab also keine Aufbewahrungsfrist — nicht „eine zu lange“, sondern gar keine.

Der Widerspruch im selben Bereich: analytics_events trägt eine Spalte ip_hash. Der Entwurf sah dort ausdrücklich einen gehashten Wert vor. Sie ist live in 0 von 4.170 Zeilen belegt — die Route schreibt wörtlich ip_hash: null. Die vier Schwestertabellen daneben legen die IP dagegen roh ab. Zwei Antworten auf dieselbe Frage, im selben Bereich, aus derselben Feder.

Behoben durch lib/perimeter/aufbewahrung.ts und /api/cron/perimeter-aufbewahrung (täglich 03:30, hinter pruefeCronGeheimnis), mit zwei Stufen statt einer:

Tabelle	IP nullen nach	Zeile löschen nach
---------	----------------	--------------------

visitors	7 Tagen	90 Tagen
visitor_locations	7 Tagen	90 Tagen
page_views	7 Tagen	90 Tagen
conversions	30 Tagen	365 Tagen
analytics_events	— (kein Rohwert)	180 Tagen

page_views trägt als einzige die Zeitspalte viewed_at statt created_at. Der Katalog führt die Spalte je Tabelle ausdrücklich mit — ein fest verdrahtetes created_at hätte hier mit 42703 abgebrochen, und weil der Lauf Fehler je Tabelle abfängt, wäre der Abbruch als „0 Zeilen“ durchgegangen.

Nur „alte Zeilen löschen“ wäre die schlechtere Lösung: sie wirft die Auswertung mit weg, obwohl an ihr nichts Personenbezogenes hängt. Die IP-Kürzung nimmt den direkten Personenbezug, die Zeile und ihre Aussage bleiben.

Woher die Zahlen kommen — und woher nicht. Das sind keine gesetzlichen Werte, und es wird hier keiner erfunden. Für Reichweitenmessung gibt es keine gesetzliche Frist; es gibt die Pflicht, eine zu haben (Art. 5 Abs. 1 lit. e DSGVO, Speicherbegrenzung). Die Werte sind eine Betriebsentscheidung, bewusst kurz und an einer Stelle änderbar.

Zwei Tabellen sind ausdrücklich ausgenommen und stehen mit Begründung in NICHT_AUTOMATISCH:

- lead_inquiries — eine Beratungsanfrage ist eine geschäftliche Willenserklärung („bitte rufen Sie mich an“), kein Messwert. Wann sie erledigt ist, entscheidet die CRM-Bearbeitung, nicht ein Kalender. Eine erfundene Frist würde offene Anfragen löschen.
- newsletter_subscribers — die abgemeldete Zeile ist der Nachweis, dass dem Widerspruch entsprochen wurde (Art. 21 DSGVO), und zugleich die Sperrliste. Sie zu löschen würde die Person erneut anschreibbar machen.

Der Lauf ist standardmäßig ein Trockenlauf. Ohne PERIMETER_AUFBEWAHRUNG_AKTIV=1 zählt er nur und ändert nichts. Der Grund steht in der Route: mit diesen Fristen wären beim ersten scharfen Lauf rund 6.900 Zeilen betroffen — Daten, die seit März liegen. Eine Frist, die beim Einschalten den halben Bestand entfernt, gehört vorher angesehen und nicht von einem nächtlichen Cron entschieden. Der Trockenlauf ist dabei kein Platzhalter: seine Zahlen sind das Entscheidungsmaterial, und npm run verify:perimeter zeigt sie ebenfalls (Prüfung B5b).

B6 (P3) — Die Newsletter-Anmeldung war ein Bestands-Orakel

POST /api/newsletter antwortete mit 409 { code: 'already_subscribed' }, wenn die Adresse schon im Verteiler stand. Damit konnte jeder von außen prüfen, ob eine bestimmte Person eingetragen ist — eine Auskunft über eine dritte Person an einen Unbekannten, über einen UNIQUE-Index auf email.

Der richtige Umgang stand seit dem 19.08.2026 nebenan in /api/auth/send-reset: dort ist eine unbekannte Adresse ausdrücklich ein Erfolg, mit dem Kommentar „kein Hinweis darauf, ob die Adresse existiert“. Zwei Wege, dieselbe Frage, zwei Antworten.

Behoben: beide Fälle antworten identisch. Der Bestand wird weiter gelesen, aber nur noch für die Entscheidung, ob eine Willkommensmail rausgeht. Die dadurch tote exists-Verzweigung in components/NewsletterSignup.tsx ist mit entfernt.

Live folgenlos: newsletter_subscribers führt 0 Zeilen — es gab nichts zu erfragen. Latent.

B7 (P3) — Der Rumpf wählte den LIKE-Präfix der Historie-Abfrage

/api/visitor-alert nimmt die gemeldete Besucher-IP aus dem Request-Rumpf. Sie wählt zweierlei: den Präfix der Historie-Abfrage (ip_address LIKE '<präfix>%') über visitor_locations) und den Schlüssel des

Stunden-Cooldowns.

Ein leerer Wert ergab `ipPrefix = ''` und damit `LIKE '%'` — die Abfrage traf jeden Besucher statt eines bestimmten. Und wer die gemeldete IP bei jedem Aufruf variiert, läuft am Cooldown vorbei; vor ihm steht dann nur noch das Aufrufer-Limit von 20/Minute, also bis zu 20 Alarmmails pro Minute in dasselbe Postfach.

Ausdrücklich benannt, was hier NICHT passiert: Das Ergebnis der Historie-Abfrage fließt in die Alarmmail an den Betreiber, nicht in die Antwort an den Aufrufer (die lautet `{ok:true}`). Es ist also keine Auskunft nach außen — es ist die falsche Abfrage und ein fremdbestimmtes Postfach.

Behoben: eine Formregel für die gemeldete IP (`istPlausibleIp`) und ein Mailbudget von 5/Stunde am Aufrufer statt an seiner Angabe. Das Budget greift erst *nach* dem Cooldown, damit der normale Betrieb — ein Besucher, eine Mail pro Stunde — es nie berührt.

Die LIKE-Escape-Behandlung (`.replace(/[%_\]/g, '\\$&')`) war bereits korrekt und blieb unverändert.

Negativbefunde — ausdrücklich mitgeteilt

Diese Punkte wurden geprüft und sind in Ordnung. Sie stehen hier, weil ein Audit, das nur Befunde nennt, nicht sagt, wie weit es geschaut hat.

N1 — anon hat nirgends Schreibrechte, RLS ist überall an. 310 public-Tabellen, davon `anon_insert=0`, `anon_update=0`, `anon_delete=0`, `rls_aus=0`. Eine ältere Notiz („anon hat DML auf viele Tabellen inkl. `audit_logs`“) ist damit überholt — der Zustand ist seither geschlossen worden.

N2 — keine SECDEF-Funktion ist für anon ausführbar. Von 362 public-Funktionen sind 244 für anon ausführbar, davon 0 mit SECURITY DEFINER. Die 244 laufen also mit den Rechten von anon und stoßen auf RLS. Der frühere Befund `cron_check_ueberfaellige_aufgaben()` ist geschlossen.

N3 — anon liest aus keiner Perimeter-Tabelle Zeilen. Sieben Tabellen mit dem öffentlichen Schlüssel abgefragt; vier antworten 200 [] (Policy filtert), drei mit 401. Zusätzlich meldet das bestehende `scripts/verify-anon-exposure.mjs` über 333 Relationen null Lecks.

N4 — der persistente Ratenzähler ist live und wirkt. `api_rate_limit_hit` ist SECURITY DEFINER mit `search_path=public`, `pg_catalog`, `api_rate_limits` hat RLS an und für anon keinerlei Grant. 70 Zeilen im Bestand über fünf Präfixe — der Zähler zählt tatsächlich. Ohne diese RPC fiel `rateLimitPersistent` auf den instanzlokalen Zähler zurück und alle Perimeter-Limits wären auf Vercel wirkungslos; deshalb steht das als eigene Prüfung im Skript.

N5 — die Webhooks prüfen ihre Signaturen fail-closed. Stripe (`constructEvent` gegen `STRIPE_WEBHOOK_SECRET`, fehlende Signatur → 400), Meta/WhatsApp (HMAC-SHA256 über den Roh-Body, Längenprüfung vor `timingSafeEqual`, fehlendes App-Secret → ablehnen statt durchlassen). Der WhatsApp-Kommentar hält ausdrücklich fest, dass hier früher `true` zurückgegeben wurde.

N6 — `/api/stripe/checkout` und `/api/stripe/portal` sind entgegen dem ersten Anschein nicht anonym. Beide rufen `requireOrgRole(orgId, ['owner','admin'])`. Sie standen nur deshalb auf der ersten Liste, weil das Guard-Grep den Wrapper nicht kannte (siehe Methodenkorrektur).

N7 — `/api/auth/send-reset` ist doppelt begrenzt und schweigt. `rateLimitPersistent` pro IP und pro Ziel-Adresse, unbekannte Adresse → `success: true`. Das ist die Vorlage, an die B6 gezogen wurde.

N8 — kein Nutzerinhalt in `dangerouslySetInnerHTML`. Alle 30 Fundstellen sind statisches JSON-LD. Der von einem Anonymen gelieferte message-Text eines Leads landet im CRM ausschließlich in React-Textknoten. Kein gespeichertes XSS über das Lead-Formular.

N9 — die Mail-Wege escapen ihre Eingaben. `/api/kontakt` und `/api/visitor-alert` führen jeden client-gelieferten Wert durch `escapeHtml`, `visitor-alert` zusätzlich durch eine CR/LF-Entfernung für den Betreff.

N10 — X-Forwarded-For ist auf Vercel nicht fälschbar (Track 10 N1, dort live belegt). Alle IP-basierten Limits dieses Tracks stehen auf dieser Grundlage.

N11 — google-reviews ist kein SSRF. Die placeld kommt aus GOOGLE_PLACE_ID, nicht aus der Anfrage; die Route nimmt überhaupt keine Parameter.

N12 — beratung-chat ist bereits gehärtet. Persistente Limits (8/Minute, 40/Stunde pro IP), ein globaler Tages-Cap gegen die Kosten, echte systemInstruction statt gefaktem User-Turn, ausdrückliche Safety-Schwellen. Der Tages-Cap ist als nicht-atomar und fail-open dokumentiert — das ist eine getroffene Entscheidung, kein übersehener Zustand.

N13 — lead_inquiries ist trotz B1 nicht lesbar. Die einzige permissive SELECT-Policy verlangt is_admin(). Ein angemeldeter Nicht-Admin bekommt null Zeilen; anon kommt nicht an die Tabelle. B1 ist ein Schreib-, kein Lesebefund — das ist der Grund für P1 statt P0.

N14 — conversions hasht bereits. E-Mail und Telefon liegen nur als SHA-256 vor. Nur die IP steht roh — und genau die ist in B5 adressiert.

Restposten — benannt und NICHT behoben

R1 — Die anon-Sperre hängt bei zwei Perimeter-Tabellen an einem Funktionsrecht, nicht an einer Policy. visitors und visitor_locations antworten anon mit 401 permission denied for function current_org_id, nicht mit 200 []. Ihre Sperre ist also der fehlende EXECUTE auf current_org_id, nicht ihre Policy. Wer dieses Recht einmal an anon zurückgibt, öffnet beide in einem Zug. Dieselbe Beobachtung wie Track 12 R2, dort für fünf Geldtabellen. Nicht behoben, weil die Antwort eine RESTRICTIVE anon_deny-Policy auf beiden Tabellen wäre — eine Schema-Änderung, die über die Fläche dieses Tracks hinausreicht und besser mit den fünf aus Track 12 zusammen entschieden wird.

R2 — kf_feature_flags trägt eine SELECT-Policy USING (true) für public. Sie wirkt heute nicht, weil anon kein SELECT-Grant auf die Tabelle hat (live geprüft: anon_S=false, 6 Zeilen). Die Sperre hängt damit wie bei R1 am Grant und nicht an der Policy. Kein Handlungsbedarf heute, aber eine Policy, deren Text mehr erlaubt als beabsichtigt.

R3 — Der erste scharfe Aufbewahrungslauf ist nicht ausgeführt. PERIMETER_AUFBEWAHRUNG_AKTIV ist nicht gesetzt; der Cron läuft als Trockenlauf. Bis zum Scharfschalten stehen die 548 bzw. 1.091 verschiedenen IP-Adressen weiter in der Datenbank. Das ist eine bewusste Übergabe, keine Erledigung — die Zahlen des ersten Laufs sollen vorher gesehen werden.

R4 — Zwei Migrationen sind eing_checked und NICHT angewendet. 20261018000000 (Policy + CHECK) und ihr Rollback 20261018000001. Bis zum Apply bleibt B1 offen; npm run verify:perimeter meldet das mit den Prüfungen B1a, B1b und B1d.

R6 — Konflikt mit einer parallel entstandenen Regel zur Migrationsbenennung. Während dieses Tracks ist aus einer anderen Sitzung docs/MIGRATION_LEDGER.md entstanden (28.08.2026, 16:03). Sie hält fest, dass die Migrationen der Tracks 9, 11 und 12 heute angewendet wurden (zuletzt 20260828125757), und stellt eine neue Regel auf: *„Neue Migrationen: ab sofort NUR mit echtem aktuellem Timestamp (YYYYMMDDHHMMSS)“*, weil die im Repo übliche fortlaufende Nummerierung in Supabase als Zukunfts-Zeitstempel ankommt.

Die beiden Migrationen dieses Tracks folgen der bisherigen Repo-Konvention (20261018000000/...0001). Das ist bewusst so und keine Missachtung: alle 404 Dateien unter supabase/migrations nutzen sie, JUENGSTE_MIGRATIONEN in lib/pilot/pre-pilot-snapshot.ts samt Regressionstest setzt die daraus folgende Sortierung voraus, und ein einzelner Ausreißer in neuem Format würde die Reihenfolge im Repo brechen, ohne das beschriebene Problem zu lösen. Die Umstellung ist eine Entscheidung für alle künftigen Migrationen und gehört nicht nebenbei in einen Sicherheits-Track. Hier nicht getroffen — der Konflikt ist benannt, damit er nicht zwischen zwei Sitzungen verloren geht.

R5 — lead_inquiries hat keine Aufbewahrungsregel. Bewusst: siehe NICHT_AUTOMATISCH. Die Entscheidung gehört in die CRM-Pflege, nicht in einen Kalender. Benannt, damit „keine Frist“ hier eine Entscheidung ist und kein zweites Vergessen.

Was geändert wurde

Migrationen (eingecheckt, NICHT angewendet, mit Rollback):

- 20261018000000_perimeter_lead_inquiries_offene_tuer.sql
- 20261018000001_rollback_perimeter_lead_inquiries_offene_tuer.sql

Neue Module:

- lib/newsletter/abmelde-token.ts — HMAC-Abmeldetoken (B3)
- lib/perimeter/aufbewahrung.ts — Aufbewahrungsentscheidungen und -lauf (B5)
- app/api/cron/perimeter-aufbewahrung/route.ts — täglicher Takt, Trockenlauf als Standard (B5)

Geänderte Routen:

- app/api/track/route.ts, app/api/track/page-view/route.ts, app/api/track-conversion/route.ts, app/api/analytics/vitals/route.ts, app/api/analytics/capi/route.ts — persistente Ratenbegrenzung (B2)
- app/api/newsletter/unsubscribe/route.ts — Token, GET/POST-Trennung, Wirkungsnachweis, Limit (B3)
- app/api/newsletter/route.ts — kein Bestands-Orakel, Abmeldelink mit Token (B3/B6)
- app/api/visitor-alert/route.ts — IP-Formregel, Mailbudget am Aufrufer (B7)

Weiteres:

- lib/dsgvo/loeschkatalog.ts + scripts/loeschkatalog-spalten.json — Einträge visitor_locations.user_id und page_views.user_id (B4)
- components/NewsletterSignup.tsx — tote exists-Verzweigung entfernt (B6)
- vercel.json — zehnter Cron-Eintrag
- lib/pilot/pre-pilot-snapshot.ts — JUENGSTE_MIGRATIONEN nachgezogen
- lib/env/register.ts — PERIMETER_AUFBEWAHRUNG_AKTIV verzeichnet
- scripts/verify-perimeter-live.mjs + npm run verify:perimeter

Prüfläufe

Lauf	Ergebnis
npm run verify:perimeter	5 von 8 bestanden, 4 Berichte
vitest run (voll)	7.968 grün, 0 rot, 38 übersprungen, 353 Dateien
npm run test:unit (node:test)	2.513 grün, 0 rot, 286 Suiten
npm run lint:forbidden	0 Treffer (24.825 Dateien)
npm run lint:route-auth	0 Treffer (413 Routen, 1.407 Dateien)
npm run lint:org-id	0 Treffer (1.422 Dateien, 190 Tabellen)
tsc --noEmit	nicht durchgelaufen — siehe unten

Die drei offenen Punkte von verify:perimeter sind genau die nicht angewendete Migration (B1a, B1b, B1d) und verschwinden mit ihrem Apply.

Zum Typecheck, ausdrücklich und ohne Beschönigung: tsc --noEmit lief lokal 25 Minuten bei rund 7 % CPU und 24 MB Speicher, ohne eine Zeile auszugeben, und wurde abgebrochen — er hat also kein Ergebnis geliefert, weder grün noch rot. Er wurde nicht ersetzt, sondern verschoben: deploy.sh führt ihn als Warnung mit, und Vercel typprüft beim Build, das heißt ein Typfehler bricht die Auslieferung dort ab. Bis dieser Build durch ist, ist der Typstand dieses Tracks ungeprüft. Die 7.968 Vitest-Fälle laufen über TypeScript-Quellen und hätten einen groben Typfehler in den geänderten Dateien mit hoher Wahrscheinlichkeit als Laufzeitfehler gezeigt — das ist ein Indiz, kein Typecheck, und wird hier auch nicht als einer ausgegeben.

Von den 7.968 grünen Fällen sind 117 neu (fünf Suiten unter __tests__/perimeter). Drei Bestandstests

wurden rot und an die Änderung gezogen, statt die Änderung zu lockern: die Fünferliste JUENGSTE_MIGRATIONEN, das ENV-Verzeichnis (die neue Variable fehlte) und die eingetragene Spaltenliste des Löschkatalogs (scripts/loeschkatalog-spalten.json, die Tatsachengrundlage der Live-Prüfung). Alle drei haben genau das gemeldet, wofür sie gebaut wurden.

Die Zaunregeln aus B2 sind nachweislich nicht leer. Gegen den Stand von HEAD geprüft: alle fünf Routen hätten die Regel rot gemeldet (rateLimit() bei zwei, eigener Modul-Zähler bei drei, fehlendes rateLimitPersistent bei allen fünf). Eine Regel, die auf dem alten Zustand grün gewesen wäre, hätte nichts bewiesen.

Einordnung zum Schluss

Der Track hat eine Fläche geprüft, die zwölf vorherige Tracks strukturell nicht sehen konnten, weil sie alle mit einem angemeldeten Akteur begonnen haben.

Das Ergebnis fiel anders aus als erwartet. Die Datenbank ist gegen den Anonymen dicht — null Schreibrechte, RLS überall, keine SECDEF-Hintertür. Die tatsächlichen Befunde liegen woanders:

- eine Policy, die für einen Aufrufer geschrieben wurde, den sie längst nicht mehr erreicht, und deshalb nur noch einem dient, an den niemand gedacht hat (B1),
- ein Umbau, der bei sieben von zwölf Routen aufhörte (B2),
- ein Recht der betroffenen Person, das an einer Adresse hing, die jeder kennt (B3),
- eine Löschung, die das Etikett nimmt und die Kennung stehen lässt (B4),
- und eine Datensammlung, für die nie jemand eine Frist entschieden hat — direkt neben einer Spalte, die zeigt, dass jemand es einmal besser vorhatte (B5).

Vier von fünf sind keine Lücken im Entwurf, sondern Reste eines früheren Entwurfs, den die Anwendung überholt hat. Das ist die Signatur dieses Tracks: am Perimeter altert Sicherheit nicht dadurch, dass jemand etwas falsch baut, sondern dadurch, dass der Weg daneben verlegt wird und die alte Tür offen stehen bleibt.